

Facultad de Ingeniería de la Universidad de Buenos Aires



Concurrencia en Sistemas Distribuidos

Materia: Programación Concurrente

Profesor/a: Damian Andres Del Campo

Integrantes

Alumno	Legajo	Mail
Sebastian Brizuela	105288	sbrizuela@fi.uba.ar
Raquel Ana Dávila	112002	radavila@fi.uba.ar
Lucas Facundo Couttulenc	109726	lcouttulenc@fi.uba.ar
Joel Isaac Fernandez Fox	104424	jfernandezf@fi.uba.ar
Luciano Costa	102104	lucosta@fi.uba.ar

1° Cuatrimestre 2026

ÍNDICE

Ejercicio 1 — Escritura con quorum	3
Escenario	3
Requisitos	3
Pista de estructura	3
Salida esperada (ejemplo de una ronda)	4
Implementación	4
Funcionamiento de las réplicas	4
Decisiones de diseño	5
Finalización anticipada al alcanzar el quórum	5
Uso de un timeout global	5
Espera de finalización de los threads (join)	6
Output	6
Ejercicio 2 — Heartbeat y detección de fallas	8
Escenario	8
Requisitos	8
Pista de estructura	8
Salida esperada (ejemplo)	9
Implementación	9
Output	10
Ejercicio 3 — Preguntas conceptuales	13

Ejercicio 1 — Escritura con quorum

Implementar una simulación de un coordinador que replica un valor en múltiples nodos y utiliza **quorum** para decidir si la escritura fue exitosa.

Escenario

- 1 thread **coordinador** envía un valor a **5 threads** réplicas.
- Cada réplica puede:
 - Confirmar exitosamente (con latencia variable entre 100ms y 800ms).
 - Fallar (no responder). La falla debe ser **aleatoria**: cada réplica tiene un 30% de probabilidad de fallar.
- El coordinador espera confirmaciones con un **timeout de 1 segundo**.
- La escritura se considera exitosa si al menos **3 de 5 réplicas** confirman (quorum = mayoría).
- Si no se alcanza el quorum, el coordinador reporta **falla de escritura**.

Requisitos

- Usar `std::sync::mpsc` para la comunicación réplicas → coordinador.
- Usar `recv_timeout` para el timeout del coordinador.
- La falla de cada réplica debe decidirse aleatoriamente al iniciar el thread (usar `rand`).
- Imprimir mensajes del flujo: envío, recepción, confirmación, timeout, resultado final.
- Ejecutar la simulación **3 veces** (loop) para ver distintos resultados según las fallas aleatorias.

Pista de estructura

```
use std::sync::mpsc;
use std::thread;
use std::time::Duration;
use rand::Rng;

const REPLICAS: usize = 5;
const QUORUM: usize = 3;
const TIMEOUT: Duration = Duration::from_secs(1);

fn main() {
    // Loop: ejecutar 3 rondas de escritura
    // En cada ronda:
    //   - Crear channel para confirmaciones
    //   - Spawn 5 threads réplicas (cada una decide aleatoriamente si falla)
    //   - Coordinador espera confirmaciones con recv_timeout
    //   - Evaluar si se alcanzó quorum
    //   - Imprimir resultado
}
```

Salida esperada (ejemplo de una ronda)

```
== Ronda 1 ==
Coordinador: enviando valor 42 a 5 réplicas...
Réplica 3: falla simulada, no respondo. 💀
Réplica 5: falla simulada, no respondo. 💀
Réplica 1: recibí valor 42, confirmando. (latencia: 230ms)
Réplica 2: recibí valor 42, confirmando. (latencia: 450ms)
Réplica 4: recibí valor 42, confirmando. (latencia: 670ms)
Coordinador: confirmación recibida de Réplica 1.
Coordinador: confirmación recibida de Réplica 2.
Coordinador: confirmación recibida de Réplica 4.
Coordinador: quorum alcanzado (3/5). Escritura EXITOSA. ✅
```

Implementación

Funcionamiento de las réplicas

Cada réplica se ejecuta en un thread independiente y simula el comportamiento de un nodo de almacenamiento distribuido. Al comenzar su ejecución, decide aleatoriamente si sufrirá una falla con una probabilidad del 30%. En caso de fallar, finaliza inmediatamente sin enviar ninguna respuesta al coordinador, simulando un nodo que deja de responder.

Si la réplica no falla, genera una latencia aleatoria entre 100 ms y 800 ms mediante una espera (sleep). Una vez transcurrido ese tiempo, imprime un mensaje indicando que recibió el valor y envía su identificador al coordinador a través de un canal mpSC, representando la confirmación de la escritura.

La decisión de determinar la falla antes de simular la latencia busca representar un nodo que se encuentra caído desde el inicio de la operación y, por lo tanto, nunca responde al coordinador.

Asimismo, la latencia se genera de forma independiente para cada réplica, permitiendo que las confirmaciones lleguen en distintos instantes y en un orden no determinista. Esto reproduce el comportamiento esperado en un sistema distribuido, donde las respuestas dependen de las condiciones de la red y de cada nodo, por lo que el orden de llegada de las confirmaciones no necesariamente coincide con el orden en que fueron creados los threads.

```
fn simular_falla() -> bool {
    let mut rng = rand::thread_rng();
    rng.gen_range(0..=100) < PROBABILIDAD_FALLA
}

fn ejecutar_replica(id: u32, valor: i32, tx: Sender<u32>) {
    let mut rng = rand::thread_rng();
    let latencia = rng.gen_range(LATENCIA_MIN_MS..=LATENCIA_MAX_MS);
    if simular_falla() {
        println!("Réplica {}: falla simulada, no respondo.", id);
        return;
    }
}
```

```

    }

    thread::sleep(Duration::from_millis(latencia));
    println!(
        "Réplica {}: recibí el valor {}, confirmando (latencia: {} ms).",
        id, valor, latencia
    );

    let _ = tx.send(id);
}

```

Decisiones de diseño

Finalización anticipada al alcanzar el quórum

El coordinador espera confirmaciones de las réplicas utilizando `recv_timeout` con un timeout máximo de un segundo para toda la operación. Sin embargo, la espera finaliza de manera anticipada cuando se alcanza el quórum (3 de 5 confirmaciones).

Esta decisión se tomó porque el objetivo del protocolo es determinar si la escritura fue exitosa. Una vez alcanzado el quórum, las confirmaciones restantes ya no modifican el resultado de la operación, por lo que continuar esperando solo incrementa la latencia sin aportar información adicional.

Además, este comportamiento refleja el funcionamiento habitual de los protocolos de replicación basados en quorum, donde la operación puede considerarse exitosa tan pronto como se obtiene la cantidad mínima de confirmaciones requerida.

Uso de un timeout global

El timeout de un segundo se interpreta como un límite para toda la operación de escritura y no como un segundo de espera entre cada confirmación recibida.

Para ello se registra el instante de inicio de la operación y, antes de cada llamada a `recv_timeout`, se calcula el tiempo restante hasta completar el segundo. De esta forma, el coordinador nunca espera más de un segundo en total, independientemente de cuándo lleguen las confirmaciones.

```

fn ejecutar_coordinador(rx: Receiver<u32>) {
    let inicio = Instant::now();
    let mut confirmaciones = 0;
    while confirmaciones < QUORUM {
        let transcurrido = inicio.elapsed();
        if transcurrido >= Duration::from_secs(TIMEOUT_SEGUNDOS) {
            break;
        }

        let restante = Duration::from_secs(TIMEOUT_SEGUNDOS) - transcurrido;
        match rx.recv_timeout(restante) {
            Ok(id) => {
                confirmaciones += 1;
                println!("Coordinador: confirmación recibida de Réplica

```

```

        {}.", id);
    }
    Err(RecvTimeoutError::Timeout) => {
        break;
    }
    Err(RecvTimeoutError::Disconnected) => {
        break;
    }
}
}

imprimir_resultado(confirmaciones);
}

```

Espera de finalización de los threads (join)

Una vez que el coordinador determina el resultado de la escritura, se realiza join sobre todos los threads de las réplicas.

Aunque el coordinador ya no necesita esperar sus respuestas para decidir el resultado, el join garantiza que todas las réplicas finalicen correctamente antes de comenzar una nueva ronda de simulación.

Esta decisión evita que queden threads ejecutándose entre rondas, impide que se mezclen mensajes pertenecientes a distintas ejecuciones y asegura una finalización ordenada del programa.

```

fn ejecutar_ronda() {
    let (tx, rx) = mpsc::channel();
    let mut handles = Vec::new();
    for id in 1..=CANTIDAD_REPLICAS {
        let tx_replica = tx.clone();
        let handle = thread::spawn(move || {
            ejecutar_replica(id, VALOR, tx_replica);
        });
        handles.push(handle);
    }

    drop(tx);
    ejecutar_coordinador(rx);
    for handle in handles {
        if handle.join().is_err() {
            eprintln!("Error: una réplica finalizó inesperadamente.");
        }
    }
}

```

Output

La salida generada por la implementación no coincide exactamente con el ejemplo provisto en el enunciado. Esto se debe a que dicho ejemplo tiene carácter ilustrativo y muestra únicamente una posible ejecución del programa, mientras que la simulación implementada

presenta un comportamiento no determinista debido al uso de latencias y fallas aleatorias en las réplicas.

Además, se realizaron algunas decisiones de diseño que modifican levemente el flujo de impresión:

- El coordinador finaliza la espera tan pronto como se alcanza el quórum de 3 confirmaciones, sin esperar respuestas adicionales. Esta decisión reduce el tiempo de respuesta de la operación y representa el comportamiento habitual de los sistemas de replicación basados en quorum, donde las confirmaciones restantes ya no afectan el resultado de la escritura.
- Luego de que el coordinador determina el resultado de la operación, el programa realiza join sobre todos los threads de las réplicas antes de iniciar la siguiente ronda. Esto garantiza que todas las réplicas finalicen correctamente, evita que se mezclen mensajes pertenecientes a distintas ejecuciones y asegura una terminación ordenada de la simulación.

En consecuencia, es posible que algunas réplicas impriman sus mensajes luego de que el coordinador haya informado el resultado de la escritura. Este comportamiento es esperable, ya que dichas réplicas continúan ejecutándose hasta completar su trabajo, aunque sus confirmaciones ya no sean necesarias para decidir el éxito o el fracaso de la operación.

Un ejemplo de salida teniendo en cuenta las consideraciones antes mencionadas son:

```
=== Ronda 1 ===
Réplica 4: falla simulada, no respondo.
Réplica 2: falla simulada, no respondo.
Réplica 3: falla simulada, no respondo.
Réplica 1: falla simulada, no respondo.
Réplica 5: recibí el valor 42, confirmando (latencia: 783 ms).
Coordinador: confirmación recibida de Réplica 5.
Coordinador: Quorum no alcanzado (1/5). Escritura fallida.

=== Ronda 2 ===
Réplica 1: falla simulada, no respondo.
Réplica 4: falla simulada, no respondo.
Réplica 3: recibí el valor 42, confirmando (latencia: 198 ms).
Coordinador: confirmación recibida de Réplica 3.
Réplica 2: recibí el valor 42, confirmando (latencia: 501 ms).
Coordinador: confirmación recibida de Réplica 2.
Réplica 5: recibí el valor 42, confirmando (latencia: 573 ms).
Coordinador: confirmación recibida de Réplica 5.
Coordinador: Quorum alcanzado (3/5). Escritura exitosa.

=== Ronda 3 ===
Réplica 4: falla simulada, no respondo.
Réplica 1: recibí el valor 42, confirmando (latencia: 440 ms).
Coordinador: confirmación recibida de Réplica 1.
Réplica 2: recibí el valor 42, confirmando (latencia: 460 ms).
Coordinador: confirmación recibida de Réplica 2.
Réplica 5: recibí el valor 42, confirmando (latencia: 505 ms).
Coordinador: confirmación recibida de Réplica 5.
Coordinador: Quorum alcanzado (3/5). Escritura exitosa.
Réplica 3: recibí el valor 42, confirmando (latencia: 680 ms).
```

Ejercicio 2 — Heartbeat y detección de fallas

Implementar un sistema de monitoreo donde un monitor detecta la caída de nodos mediante heartbeats.

Escenario

- **3 threads nodos** que envían heartbeats periódicos (cada 500ms) a un thread **monitor**.
- El **monitor** espera heartbeats de cada nodo. Si no recibe heartbeat de un nodo en 1.5 segundos, lo declara caído.
- Después de un tiempo aleatorio (entre 2 y 4 segundos), **uno de los nodos deja de enviar heartbeats** (simula crash).
- Otro nodo puede "volverse lento" (enviar heartbeats cada 1.2 segundos en vez de 500ms) simulando una red congestionada.
- El monitor debe reportar el estado de cada nodo: ACTIVO, SOSPECHOSO (un timeout), CAÍDO (dos timeouts consecutivos).

Requisitos

- Usar mpsc channels para los heartbeats (cada nodo envía su ID periódicamente).
- El monitor usa `recv_timeout` para detectar ausencia de heartbeats.
- Mantener un registro del último heartbeat de cada nodo.
- Distinguir entre un nodo caído y un nodo lento (al menos intentar).
- El programa debe correr por 8 segundos y luego imprimir un reporte final del estado de cada nodo.
- El programa debe terminar limpiamente (sin colgarse).

Pista de estructura

```
use std::sync::mpsc;
use std::thread;
use std::time::{Duration, Instant};
use std::collections::HashMap;

enum EstadoNodo {
    Activo,
    Sospechoso,
    Caído,
}

fn main() {
    // Crear channel compartido para heartbeats (todos los nodos -> monitor)
    // Spawn 3 nodos con comportamiento distinto:
    //   - Nodo 1: normal (heartbeat cada 500ms, siempre)
    //   - Nodo 2: se vuelve lento después de 3s (heartbeat cada 1200ms)
    //   - Nodo 3: muere después de 2-4s (deja de enviar)
    // Monitor: loop con recv_timeout, actualizar estado de nodos
    // Después de 8s: imprimir reporte y terminar
}
```

Salida esperada (ejemplo)

```
[0.5s] Monitor: heartbeat de Nodo 1. Estado: ACTIVO
[0.5s] Monitor: heartbeat de Nodo 2. Estado: ACTIVO
[0.5s] Monitor: heartbeat de Nodo 3. Estado: ACTIVO
...
[3.2s] Monitor: Nodo 3 sin heartbeat hace 1.5s. Estado: SOSPECHOSO ⚠️
[4.7s] Monitor: Nodo 3 sin heartbeat hace 3.0s. Estado: CAÍDO 💀
[4.0s] Monitor: Nodo 2 heartbeat tardío (1.1s). Estado: SOSPECHOSO ⚠️
[4.6s] Monitor: heartbeat de Nodo 2. Estado: ACTIVO (recuperado)
...

== Reporte final (8s) ==
Nodo 1: ACTIVO (último heartbeat hace 0.3s)
Nodo 2: ACTIVO (último heartbeat hace 0.9s) – tuvo episodios de lentitud
Nodo 3: CAÍDO (último heartbeat hace 5.2s) – declarado caído en t=4.7s
```

Implementación

La simulación se implementó utilizando un hilo por cada nodo y un hilo principal que actúa como monitor. La comunicación entre ellos se realizó mediante un canal `mpsc`, donde cada nodo envía periódicamente un *heartbeat* al monitor.

Para representar el estado de cada nodo se definieron dos enums:

```
enum EstadoNodo {
    Activo,
    Sospechoso,
    Caído,
}

enum ComportamientoNodo {
    Normal,
    Variable,
    Crash,
}
```

Cada nodo mantiene un comportamiento distinto según lo solicitado por la consigna:

- **Nodo 1:** envía un heartbeat exactamente cada 500 ms, por lo que permanece siempre en estado **Activo**.
- **Nodo 2:** antes de cada heartbeat elige aleatoriamente un intervalo de **500 ms, 700 ms o 1200 ms**, simulando retrasos temporales de red. Esto provoca que el monitor lo marque como **Sospechoso** cuando supera los 500 ms, recuperándose al recibir el siguiente heartbeat.
- **Nodo 3:** comienza enviando heartbeats normalmente y, en un instante aleatorio entre los **2 y 4 segundos**, deja de enviar mensajes para simular una caída definitiva.

El monitor almacena para cada nodo su último heartbeat y su estado actual:

```
struct InfoNodo {
    estado: EstadoNodo,
```

```
ultimo_heartbeat: Instant,  
}
```

Cada 100 ms revisa el tiempo transcurrido desde el último heartbeat recibido y actualiza el estado según los límites definidos por la consigna:

```
let nuevo_estado = if tiempo >= Duration::from_millis(LIMITE_CAIDO_MS) {  
    EstadoNodo::Caído  
} else if tiempo > Duration::from_millis(LIMITE_ACTIVADO_MS) {  
    EstadoNodo::Sospechoso  
} else {  
    EstadoNodo::Activo  
};
```

Para evitar que pequeños retrasos del sistema operativo generaran falsos positivos (por ejemplo, un heartbeat de 500 ms recibido a los 502 ms), se incorporó un margen de tolerancia de **20 ms** al momento de calcular el tiempo transcurrido.

Además, los envíos periódicos de heartbeat no se implementaron mediante `sleep(500 ms)` repetidos, ya que esos pequeños errores de temporización se acumulan con el tiempo. En su lugar, cada nodo calcula el instante absoluto del próximo envío (`proximo_heartbeat`) y duerme únicamente hasta ese momento, manteniendo así una periodicidad estable durante toda la simulación.

Output

```
[0.0s] Nodo 1: heartbeat inicial.  
[0.0s] Monitor: Heartbeat recibido de Nodo 1.  
[0.0s] Nodo 3: heartbeat inicial.  
[0.0s] Nodo 2: heartbeat inicial.  
[0.0s] Monitor: Heartbeat recibido de Nodo 3.  
[0.0s] Monitor: Heartbeat recibido de Nodo 2.  
[0.5s] Nodo 1: heartbeat enviado (500 ms).  
[0.5s] Monitor: Heartbeat recibido de Nodo 1.  
[0.5s] Monitor: Heartbeat recibido de Nodo 2.  
[0.5s] Monitor: Heartbeat recibido de Nodo 3.  
[0.5s] Nodo 2: heartbeat enviado (500 ms).  
[0.5s] Nodo 3: heartbeat enviado (500 ms).  
[1.0s] Nodo 3: heartbeat enviado (500 ms).  
[1.0s] Nodo 1: heartbeat enviado (500 ms).  
[1.0s] Monitor: Heartbeat recibido de Nodo 3.  
[1.0s] Monitor: Heartbeat recibido de Nodo 1.  
[1.1s] Monitor: Nodo 2 sin heartbeat hace 0.60s. Estado: SOSPECHOSO.  
[1.2s] Nodo 2: heartbeat enviado (700 ms).  
[1.2s] Monitor: Heartbeat recibido de Nodo 2. Estado: ACTIVO (recuperado).  
[1.5s] Nodo 3: heartbeat enviado (500 ms).
```

```
[1.5s] Nodo 1: heartbeat enviado (500 ms).
[1.5s] Monitor: Heartbeat recibido de Nodo 3.
[1.5s] Monitor: Heartbeat recibido de Nodo 1.
[1.7s] Nodo 2: heartbeat enviado (500 ms).
[1.7s] Monitor: Heartbeat recibido de Nodo 2.
[2.0s] Nodo 3: heartbeat enviado (500 ms).
[2.0s] Nodo 1: heartbeat enviado (500 ms).
[2.0s] Monitor: Heartbeat recibido de Nodo 3.
[2.0s] Monitor: Heartbeat recibido de Nodo 1.
[2.3s] Monitor: Nodo 2 sin heartbeat hace 0.59s. Estado: SOSPECHOSO.
[2.5s] Nodo 3: heartbeat enviado (500 ms).
[2.5s] Monitor: Heartbeat recibido de Nodo 3.
[2.5s] Monitor: Heartbeat recibido de Nodo 1.
[2.5s] Nodo 1: heartbeat enviado (500 ms).
[2.9s] Nodo 2: heartbeat enviado (1200 ms).
[2.9s] Monitor: Heartbeat recibido de Nodo 2. Estado: ACTIVO (recuperado).
[3.0s] Nodo 3: heartbeat enviado (500 ms).
Nodo 3: CRASH.
[3.0s] Nodo 3: finalizado.
[3.0s] Nodo 1: heartbeat enviado (500 ms).
[3.0s] Monitor: Heartbeat recibido de Nodo 3.
[3.0s] Monitor: Heartbeat recibido de Nodo 1.
[3.4s] Nodo 2: heartbeat enviado (500 ms).
[3.4s] Monitor: Heartbeat recibido de Nodo 2.
[3.5s] Nodo 1: heartbeat enviado (500 ms).
[3.5s] Monitor: Heartbeat recibido de Nodo 1.
[3.6s] Monitor: Nodo 3 sin heartbeat hace 0.59s. Estado: SOSPECHOSO.
[3.9s] Nodo 2: heartbeat enviado (500 ms).
[3.9s] Monitor: Heartbeat recibido de Nodo 2.
[4.0s] Nodo 1: heartbeat enviado (500 ms).
[4.0s] Monitor: Heartbeat recibido de Nodo 1.
[4.4s] Monitor: Nodo 2 sin heartbeat hace 0.52s. Estado: SOSPECHOSO.
[4.5s] Nodo 1: heartbeat enviado (500 ms).
[4.5s] Monitor: Heartbeat recibido de Nodo 1.
[4.6s] Monitor: Nodo 3 sin heartbeat hace 1.58s. Estado: CAIDO.
[5.0s] Nodo 1: heartbeat enviado (500 ms).
[5.0s] Monitor: Heartbeat recibido de Nodo 1.
[5.1s] Nodo 2: heartbeat enviado (1200 ms).
[5.1s] Monitor: Heartbeat recibido de Nodo 2. Estado: ACTIVO (recuperado).
[5.5s] Nodo 1: heartbeat enviado (500 ms).
[5.5s] Monitor: Heartbeat recibido de Nodo 1.
[5.7s] Monitor: Nodo 2 sin heartbeat hace 0.60s. Estado: SOSPECHOSO.
```

```
[6.0s] Nodo 1: heartbeat enviado (500 ms).
[6.0s] Monitor: Heartbeat recibido de Nodo 1.
[6.3s] Nodo 2: heartbeat enviado (1200 ms).
[6.3s] Monitor: Heartbeat recibido de Nodo 2. Estado: ACTIVO (recuperado).
[6.5s] Nodo 1: heartbeat enviado (500 ms).
[6.5s] Monitor: Heartbeat recibido de Nodo 1.
[6.8s] Monitor: Nodo 2 sin heartbeat hace 0.50s. Estado: SOSPECHOSO.
[7.0s] Nodo 1: heartbeat enviado (500 ms).
[7.0s] Monitor: Heartbeat recibido de Nodo 1.
[7.5s] Nodo 2: heartbeat enviado (1200 ms).
[7.5s] Nodo 1: heartbeat enviado (500 ms).
[7.5s] Monitor: Heartbeat recibido de Nodo 2. Estado: ACTIVO (recuperado).
[7.5s] Monitor: Heartbeat recibido de Nodo 1.
[8.0s] Nodo 1: finalizado.
[8.0s] Monitor: Nodo 1 sin heartbeat hace 0.52s. Estado: SOSPECHOSO.
[8.0s] Monitor: Nodo 2 sin heartbeat hace 0.52s. Estado: SOSPECHOSO.

===== REPORTE FINAL =====
Nodo 1 -> SOSPECHOSO | Último heartbeat hace 516 ms
Nodo 2 -> SOSPECHOSO | Último heartbeat hace 517 ms
Nodo 3 -> CAIDO      | Último heartbeat hace 5016 ms
[8.7s] Nodo 2: finalizado.

Simulación finalizada.
```

Ejercicio 3 — Preguntas conceptuales

Responder las siguientes preguntas (extensión sugerida: 3-5 líneas cada una):

- A.** ¿Por qué los ejercicios 1 y 2 NO son sistemas distribuidos reales? Mencionar al menos 3 garantías que tiene mpvc que una red real no tiene.

RESPUESTA:

En sistemas distribuidos reales para que haya comunicación entre los nodos lo hacen mediante un sistema de mensajes en la red física. En ambos ejercicios, mpvc nos permite simular ese sistema corriendo en memoria local compartida ofreciéndonos 3 garantías sobre la red:

- Los mensajes siempre llegan, a comparación con la red donde los mensajes se pueden perder, duplicarse o llegar tarde.
- Al ser concurrencia local los mensajes tardan nanosegundos en llegar. El viaje por el internet depende a qué distancia están los nodos y sobre qué medio se transportan llegando a durar en viaje milisegundos o incluso segundos.
- El orden es determinista, mientras que en la red el orden depende de qué mensaje viaja más rápido causando a veces que un mensaje que se envió después puede llegar antes que uno que se envió antes, mpvc se maneja con un orden FIFO estricto por emisor asegurando que los mensajes de un mismo nodo lleguen exactamente en el orden en que se enviaron, algo que los routers de una red real no garantizan.

- B.** Quorum y CAP: En el ejercicio 1, si bajamos el quorum a 1 (confirmar con solo 1 réplica), ¿qué ganamos y qué perdemos en términos del teorema CAP?

RESPUESTA:

Al bajar el quórum a 1, ganamos Disponibilidad (A) y minimizamos la latencia, ya que el sistema completará la escritura con éxito incluso si fallan 4 de las 5 réplicas. Por el contrario, perdemos Consistencia (C), puesto que las réplicas restantes quedarán desactualizadas inmediatamente, permitiendo que clientes concurrentes lean datos viejos o divergentes al consultar distintos nodos del sistema.

- C.** Falsos positivos: En el ejercicio 2, el nodo 2 se vuelve lento pero sigue vivo. En un sistema real, ¿qué consecuencias puede tener declarar "caído" a un nodo que en realidad está vivo? Dar un ejemplo concreto.

RESPUESTA:

En un sistema asíncrono, definir un timeout demasiado corto genera falsos positivos debido a fallas de temporización o redes lentas. La consecuencia de declarar "caído" a un nodo vivo es que se desencadenan innecesariamente costosos problemas de sincronización global para reorganizar el sistema.

- D.** El problema de los dos generales: Expliquen por qué el coordinador del ejercicio 1 nunca puede tener certeza absoluta de que una réplica recibió el valor, incluso si recibe la confirmación. Relacionar con el problema visto en clase.

RESPUESTA:

Si lleváramos la simulación más allá haciendo que la latencia de confirmación sea más grande que un segundo, puede llegarse a dar que cuando se escriba el mismo valor en distintas rondas el orquestador reciba un falso positivo, una confirmación de una ronda anterior que recién llegó y la puede tomar como confirmación para la ronda actual. Por lo tanto, es imposible lograr acuerdo con certeza absoluta porque los mensajes pueden perderse o tardar más de la cuenta como lo plantea el problema de los dos generales.